ackee
blockchain security

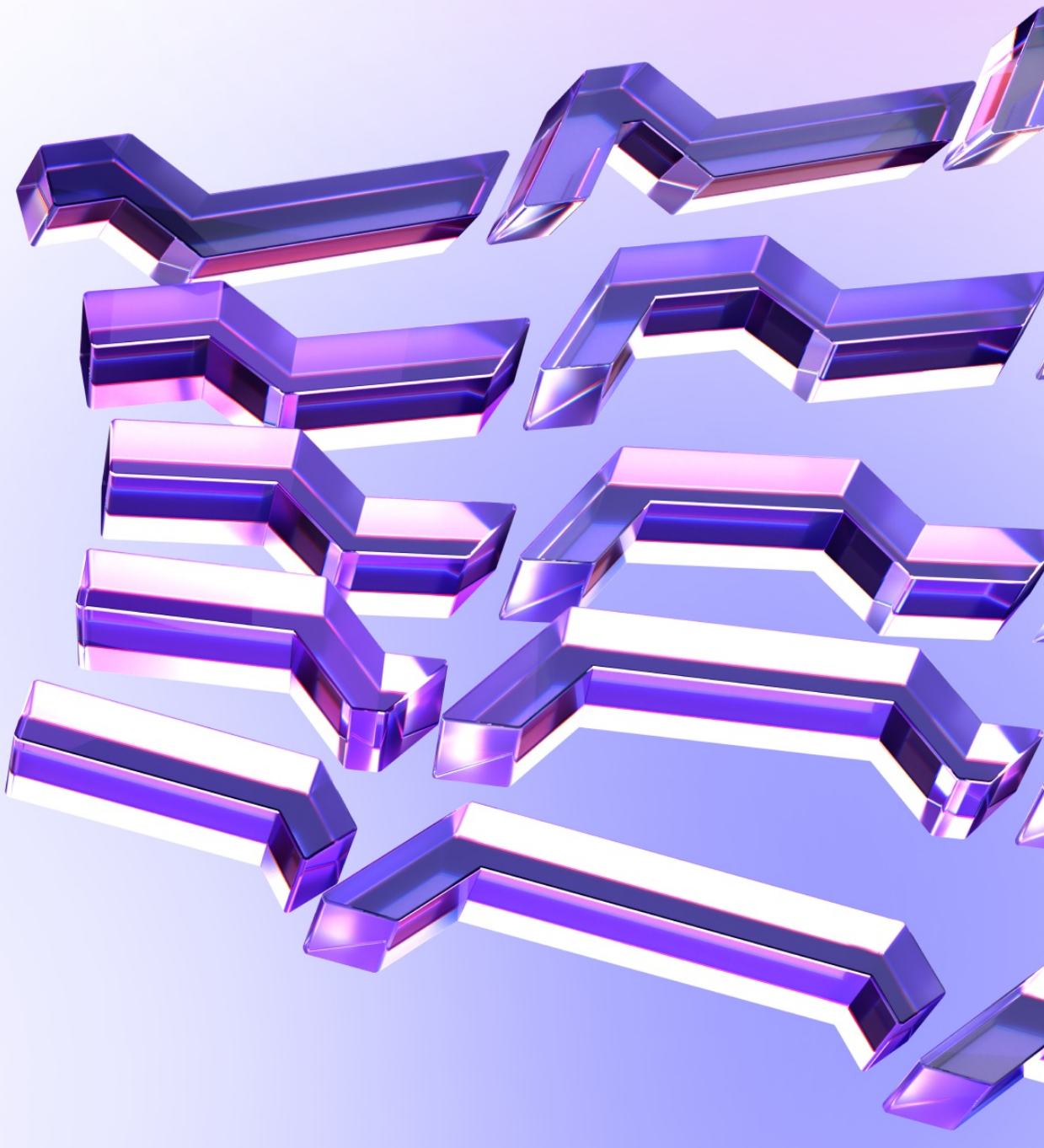
Wake

CMTA

Rules

March 16, 2026 18:00 UTC

Wake Arena AI Report



Contents

1. Document Revisions

1.0	AI Scan	Mar 16, 2026 18:00 UTC

2. Overview

This document presents the findings Wake Arena identified using AI vulnerability checks in the reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling Wake for Ethereum and Trident for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the School of Solana and the Solana Auditors Bootcamp.

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.
Rohanske nabrezi 717/4
186 00 Prague, Czech Republic
https://ackee.xyz
hello@ackee.xyz

2.2. Wake Arena

This report was created by Wake Arena, an automated vulnerability analysis tool. It uses Wake Framework with additional detectors for AI and static analysis of Solidity smart contracts.

To find potential vulnerabilities, Wake uses:

- The code structure and patterns
- Control flow graphs
- Data flow graphs
- Common vulnerability patterns
- Contract interactions

Wake Arena aims for precision, not recall. This means the presented findings are optimized for true positives, not for finding all possible issues, and should always be complemented with additional manual review for a complete security assessment.

2.3. Disclaimer

We've put our best effort to find known vulnerabilities in the system, but automated findings shouldn't be considered a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

2.4. Finding Classification

Each finding is classified by two independent ratings: *Impact* and *Confidence*.

Impact

Measuring the potential consequences of the issue on the system.

- **Critical** - Code that activates the issue directly enables theft of significant assets with minimal preconditions.
- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue represents a potential security concern in the code structure or logic that could become problematic with code modifications.
- **Info** - The issue relates to code quality practices that may affect security. Examples include insufficient logging for critical operations or inconsistent error handling patterns.

Confidence

Indicating the probability that the identified issue is a valid security concern.

- **High** - The analysis has identified a pattern that strongly indicates the presence of the issue.
- **Medium** - Evidence suggests the issue exists, but manual verification is recommended.
- **Low** - Potential indicators of the issue have been detected, but there is a significant possibility of false positives.

3. Executive Summary

Revision 1.0

Audit Overview

This audit of CMTA/Rules at commit `d72a98abbba29cd82a7056b59104e82ac65389e7` reviewed the modular compliance and rule engine implementing ERC-1404, ERC-3643, ERC-2980, and ERC-7551 style restrictions. The scope covered operational rule `RuleConditionalTransferLight`, multiple validation rules (whitelist, blacklist, sanctions, identity), the binding layer in `ERC3643ComplianceModule`, and the custom access control module `AccessControlModuleStandalone`.

A total of 4 AI-detected issues were identified across the codebase (high: 1, medium: 1, info: 2):

- High: Cross-token approval replay and approval draining in `RuleConditionalTransferLight` due to approvals keyed only by `from`, `to`, `value` and not scoped by the calling token. Storage `approvalCounts` and helper ` _transferHash` in `src/rules/operation/abstract/RuleConditionalTransferLightBase.sol` ignore token identity, while `_authorizeTransferExecution` in `src/rules/operation/RuleConditionalTransferLight.sol` permits any bound token via `isTokenBound`. With `ERC3643ComplianceModule` allowing multiple bound tokens, approvals can be reused across tokens or consumed after rebinding.
- Medium: Incomplete `supportsInterface` implementations break ERC-165-based discovery. `RuleTransferValidation.supportsInterface` does not acknowledge `IERC165` or the compliance interfaces it implements; `RuleConditionalTransferLight.supportsInterface` exposes only `IRule` and what `AccessControlEnumerable` provides, omitting `IERC1404`, `IERC1404Extend`,

`IERC3643ComplianceRead`, and `IERC7551Compliance`.

- Info: `AccessControlModuleStandalone.hasRole` treats `DEFAULT_ADMIN_ROLE` as having every role, diverging from OpenZeppelin semantics and making `grantRole`, `revokeRole`, `renounceRole`, and enumeration misleading for admins.
- Info: Documentation for `RuleERC2980` omits that a frozen `spender` blocks `transferFrom`, while code in `RuleERC2980Base._detectTransferRestrictionFrom` enforces this, risking integrator confusion.

Overall, the codebase is well-structured, heavily composes OpenZeppelin libraries, and cleanly separates operational and validation rules. However, the high-severity approval scoping flaw in `RuleConditionalTransferLight` is a concrete, exploitable logic issue, and the ERC-165 introspection gaps can cause integration failures or silent bypasses. Security posture is moderate and requires targeted remediations before production deployment.

Key Technical Findings

- High: Cross-token approval replay in `RuleConditionalTransferLight`. Approvals are stored in `approvalCounts` keyed by `_transferHash(from, to, value)` without token scoping (`src/rules/operation/abstract/RuleConditionalTransferLightBase.sol`). Execution is authorized for any bound token via `isTokenBound` in `_authorizeTransferExecution` (`src/rules/operation/RuleConditionalTransferLight.sol`), and the binding layer maintains multiple tokens (`lib/RuleEngine/src/modules/ERC3643ComplianceModule.sol`). Recommendation: include the token address in the approval key (per-token mapping), add `approveTransferFor token` or enforce single-token binding, and clear per-token approvals on unbind.

- Medium: Incomplete ERC-165 reporting. `RuleTransferValidation.supportsInterface` acknowledges only `IRule` and does not return true for `IERC165`, `IERC1404`, `IERC1404Extend`, `IERC3643ComplianceRead`, `IERC7551Compliance`. `RuleConditionalTransferLight.supportsInterface` omits these compliance IDs as well. Recommendation: implement `supportsInterface` to OR all relevant interface IDs across bases and explicitly return true for the compliance interfaces actually implemented.
- Info: `AccessControlModuleStandalone.hasRole` makes `DEFAULT_ADMIN_ROLE` pass all role checks, creating divergence between effective privileges and `_roles` state and events, and bypassing granular revocation. Recommendation: remove the override and introduce `onlyRoleOrAdmin` where intended, or override role mutation functions to avoid misleading events and enumeration.
- Info: `RuleERC2980` documentation inconsistency regarding frozen `spender` vs code in `RuleERC2980Base._detectTransferRestrictionFrom` that returns `CODE_ADDRESS_SPENDER_IS_FROZEN`. Recommendation: update README and tables to include the spender case and add tests demonstrating spender freeze on `transferFrom`.

Conclusion

The project demonstrates solid modular design and extensive reuse of proven libraries, but the detected high-severity logic flaw in `RuleConditionalTransferLight` and the medium-severity ERC-165 introspection gaps materially affect correctness and integrations. In its current state, the codebase is not ready for deployment.

Remediating approval scoping to the token level and fixing `supportsInterface` to accurately report implemented compliance interfaces, followed by focused tests, will bring the system to a production-ready posture.

4. Findings Summary

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
0	1	1	0	0	2	**4**

Table 1. Findings Count by Impact

Findings in detail:

Finding title	Impact	Reported	Status
H1: ConditionalTransferLight approvals not scoped by token enable cross-token replay and approval draining	**High**	1.0	Reported
M1: Incomplete supportsInterface breaks ERC-165 and compliance interface discovery in rule contracts	Medium	1.0	Reported
I1: RuleERC2980 docs omit spender freeze; code blocks transferFrom when spender is frozen	Info	1.0	Reported
I2: hasRole override: DEFAULT_ADMIN_ROLE implicitly passes all role checks, making revoke/renounce ineffective and events misleading	Info	1.0	Reported

Table 2. Table of Findings

Report Revision 1.0

Findings

The following section presents the list of findings discovered in this revision. For the complete list of all findings, Go back to Findings Summary

H1: ConditionalTransferLight approvals not scoped by token enable cross-token replay and approval draining

High impact finding

Impact:	High	Confidence:	High
Target:	ERC3643ComplianceModule.sol, RuleConditionalTransferLight.sol, RuleConditionalTransferLightBase.sol	Type:	Logic error

Description

The rule stores and consumes operator approvals in `approvalCounts` keyed only by `_transferHash(from, to, value)`. The bound-token identity is never included in the key. Execution authorization only checks that `msg.sender` is within the bound-token set, not which specific token an approval belongs to. Because the binding module allows multiple simultaneous token bindings and rebinding over time, an approval granted for Token A can be consumed by a transfer on Token B that uses the same `(from, to, value)` tuple, enabling cross-token replay and deliberate approval draining.

Listing 1. Code snippet from src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

```
31 mapping(bytes32 => uint256) public approvalCounts;
32
33 function approveTransfer(
34     address from,
35     address to,
36     uint256 value
37 ) public onlyTransferApprover {
```

```
38     bytes32 transferHash = _transferHash(from, to, value);
39     approvalCounts[transferHash] += 1;
40     emit TransferApproved(from, to, value, approvalCounts[transferHash]);
41 }
```

Listing 2. Code snippet from src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

```
122 function detectTransferRestriction(
123     address from,
124     address to,
125     uint256 value
126 ) public view override(IERC1404) returns (uint8) {
127     if (from == address(0) || to == address(0)) {
128         return uint8(IERC1404Extend.REJECTED_CODE_BASE.TRANSFER_OK);
129     }
130     bytes32 transferHash = _transferHash(from, to, value);
131     if (approvalCounts[transferHash] == 0) {
132         return CODE_TRANSFER_REQUEST_NOT_APPROVED;
133     }
134     return uint8(IERC1404Extend.REJECTED_CODE_BASE.TRANSFER_OK);
135 }
```

Listing 3. Code snippet from src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

```
195 function _transferred(
196     address from,
197     address to,
198     uint256 value
199 ) internal virtual {
200     if (from == address(0) || to == address(0)) {
201         return;
202     }
203     bytes32 transferHash = _transferHash(from, to, value);
204     uint256 count = approvalCounts[transferHash];
205 
206     require(count != 0, TransferNotApproved());
207 
208     approvalCounts[transferHash] = count - 1;
209     emit TransferExecuted(from, to, value, approvalCounts[transferHash]);
210 }
```

Listing 4. Code snippet from src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

```
212 function _transferHash(
213     address from,
214     address to,
215     uint256 value
216 ) internal pure virtual returns (bytes32 hash) {
217     // Linter suggestion (`asm-keccak256`): hash packed values in assembly
    to avoid abi.encodePacked overhead.
218     assembly ("memory-safe") {
219         let ptr := mload(0x40)
220         mstore(ptr, shl(96, from))
221         mstore(add(ptr, 0x20), shl(96, to))
222         mstore(add(ptr, 0x40), value)
223         hash := keccak256(ptr, 0x60)
224     }
225 }
```

Listing 5. Code snippet from src/rules/operation/RuleConditionalTransferLight.sol

```
58 function _authorizeTransferExecution() internal view virtual override {
59     require(
60         isTokenBound(_msgSender()),
61         RuleConditionalTransferLight_TransferExecutorUnauthorized(
62             _msgSender()
63         )
64     );
65 }
```

Listing 6. Code snippet from lib/RuleEngine/src/modules/ERC3643ComplianceModule.sol

```
16 EnumerableSet.AddressSet private _boundTokens;
```

Listing 7. Code snippet from lib/RuleEngine/src/modules/ERC3643ComplianceModule.sol

```
60 function isTokenBound(
61     address token
```

```
62 ) public view virtual override returns (bool) {
63     return _boundTokens.contains(token);
64 }
```

Realistic impact in production:

- Cross-token approval replay: When a single `RuleConditionalTransferLight` instance is bound to two tokens, any approval for `(from, to, value)` created for Token A can be reused on Token B, authorizing an unintended transfer on a different asset.
- Approval draining/DoS: A malicious operator of Token B can deliberately execute a matching transfer to consume approvals intended for Token A, causing the intended Token A transfer to fail later with `TransferNotApproved`.
- Rebinding hazard: If the rule is unbound from Token A and later bound to Token B, stale approvals left in `approvalCounts` can be consumed on Token B.

Exploit Scenario

Actors and setup:

- Alice: issuer/operator who approved a transfer on Token A.
- Bob: recipient on both Token A and Token B.
- Carol: malicious controller of Token B.
- R: a single `RuleConditionalTransferLight` instance bound to both Token A and Token B via the compliance module.

Attack steps:

1. Alice approves a transfer on Token A by calling `approveTransfer(Alice, Bob, 100)` on R. This increments `approvalCounts[keccak256(Alice, Bob, 100)]`.

2. Carol, operating Token B (also bound to R), initiates a transfer `transferFrom(Alice, Bob, 100)` on Token B.
3. Token B calls into R. Authorization passes because `_authorizeTransferExecution` only checks `isTokenBound(msg.sender)`.
4. R computes the same `_transferHash(Alice, Bob, 100)` and finds a positive approval count. The transfer is allowed and `_transferred` consumes the approval.
5. Later, when Alice attempts the intended transfer on Token A, R now reports `CODE_TRANSFER_REQUEST_NOT_APPROVED` or reverts with `TransferNotApproved` because the approval was already consumed by Token B.

Variant (rebinding):

- Alice approves on Token A, then the admin unbinds Token A and binds Token B to R. The stale approval in `approvalCounts` remains and can be used for a transfer on Token B.

Recommendation

Scope approvals by token identity and, if applicable, clear or segregate approvals on (re)binding.

Concrete hardening options:

- Include the bound token address in the approval key. For example, replace `approvalCounts[hash(from,to,value)]` with `approvalCountsByToken[token][hash(from,to,value)]`, where `token` is the calling token `msg.sender` during consumption and an explicit parameter during approval. Update `_transferHash` (or add `_scopedTransferHash`) to incorporate `token`.
- Add `approveTransferFor(address token, address from, address to, uint256 value)` that records approvals under the specified `token`. Keep

`approveTransfer` only if the rule enforces a single bound token and resolves `token` via `getTokenBound()`.

- On `unbindToken(token)`, optionally purge or invalidate that token's approvals to prevent stale reuse after rebinding.
- If multi-token binding is not required, enforce a single-token model in the compliance module (reject a second bind) so approvals cannot be shared across assets.

Add tests that bind two tokens to the same rule and assert that approvals recorded for Token A are not visible/consumable for Token B.

Go back to Findings Summary

M1: Incomplete supportsInterface breaks ERC-165 and compliance interface discovery in rule contracts

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	RuleConditionalTransferLight.sol, RuleIdentityRegistryOwnable2Step.sol, RuleTransferValidation.sol	Type:	Standards violation

Description

Several rule contracts implement methods from ERC-1404/3643/7551 compliance interfaces but do not report those interfaces in `supportsInterface`. In `RuleTransferValidation`, even ERC-165 (0x01ffc9a7) is not acknowledged, violating EIP-165. Operational rule `RuleConditionalTransferLight` also omits compliance interface IDs, delegating only to `AccessControlEnumerable`.

Listing 8. Code snippet from src/rules/validation/abstract/core/RuleTransferValidation.sol

```
108 function supportsInterface(
109     bytes4 interfaceId
110 ) public view virtual returns (bool) {
111     return
112         interfaceId == type(IRule).interfaceId ||
113         interfaceId == RuleInterfaceId.IRULE_INTERFACE_ID;
114 }
```

Listing 9. Code snippet from src/rules/operation/RuleConditionalTransferLight.sol

```
27 function supportsInterface(
28     bytes4 interfaceId
29 )
30     public
31     view
32     virtual
33     override(AccessControlEnumerable, IERC165)
34     returns (bool)
35 {
36     return
37         interfaceId == RuleInterfaceId.IRULE_INTERFACE_ID ||
38         interfaceId == type(IRule).interfaceId ||
39         AccessControlEnumerable.supportsInterface(interfaceId);
40 }
```

Listing 10. Code snippet from src/rules/validation/deployment/RuleIdentityRegistryOwnable2Step.sol

```
12 contract RuleIdentityRegistryOwnable2Step is
13     RuleIdentityRegistryBase,
14     Ownable2Step
15 {
16     constructor(
17         address owner,
18         address identityRegistry_
19     ) RuleIdentityRegistryBase(identityRegistry_) Ownable(owner) {}
```

Impact in production:

- ERC-165–based discovery (e.g., via `ERC165Checker`) fails to recognize that these rules implement `IERC1404`, `IERC1404Extend`, `IERC3643ComplianceRead`, and `IERC7551Compliance`. For `RuleTransferValidation`-based rules, even `IERC165` appears unsupported.
- Integrations that verify rule capabilities via introspection may reject deployment/configuration or silently skip calling compliance hooks, degrading or disabling enforcement.

Exploit Scenario

Actors and setup:

- Alice: token issuer integrating rules via a UI and on-chain manager.
- Bob: integration tool that validates rule capabilities using `ERC165Checker`.
- Rv: a validation rule inheriting `RuleTransferValidation` (e.g., identity, sanctions).
- Ro: `RuleConditionalTransferLight` operational rule.

Attack/impact scenarios:

1. Alice selects Rv in Bob's configuration UI. Bob calls `supportsInterface(0x01ffc9a7)` and `supportsInterface(type(IERC3643ComplianceRead).interfaceId)`. Both return `false`, so Bob rejects Rv as a valid rule. Alice cannot configure enforcement (configuration DoS).
2. Alice deploys Ro and expects consumers to call its `IERC1404` /`IERC7551Compliance` hooks. Another dApp checks `ERC165` for these interfaces. Because Ro does not report them (only `IRule` and `AccessControlEnumerable`), the dApp skips calling the hooks, and transfers proceed without expected compliance checks (silent enforcement bypass at integration boundary).

Recommendation

Make `supportsInterface` ERC-165 compliant and report all actually implemented compliance interfaces at the base-class level, then compose via `super` in subclasses.

Concrete steps:

- In `RuleTransferValidation`, return `true` for:

- `type(IERC165).interfaceId`
- `type(IERC1404).interfaceId` and `type(IERC1404Extend).interfaceId`
- `type(IERC3643ComplianceRead).interfaceId`
- `type(IERC7551Compliance).interfaceId`
- existing `IRule`/`IRULE_INTERFACE_ID`
- In `RuleConditionalTransferLight`, also return `true` for the compliance interfaces it implements in `RuleConditionalTransferLightBase`: `IERC1404`, `IERC1404Extend`, `IERC3643ComplianceRead`, `IERC7551Compliance` (in addition to delegating to `AccessControlEnumerable`).
- When multiple inheritance is involved, implement `supportsInterface` using logical OR over all base classes' support and explicitly add any missing interface IDs.
- Add tests using `ERC165Checker` to assert `true` for all expected interface IDs on each rule contract.

Go back to Findings Summary

I1: RuleERC2980 docs omit spender freeze; code blocks transferFrom when spender is frozen

Impact:	Info	Confidence:	High
Target:	AGENTS.md, CLAUDE.md, README.md, RuleERC2980Base.sol	Type:	Code quality

Description

`RuleERC2980Base` explicitly blocks `transferFrom`-style transfers when the spender is frozen (`CODE_ADDRESS_SPENDER_IS_FROZEN = 62`). Several documents describe the frozenlist as affecting only sender/recipient, which can mislead integrators and cause unexpected failures.

Listing 11. Code snippet from src/rules/validation/abstract/base/RuleERC2980Base.sol

```
73 function _detectTransferRestrictionFrom(
74     address spender,
75     address from,
76     address to,
77     uint256 value
78 ) internal view virtual override returns (uint8) {
79     if (_isFrozen(spender)) {
80         return CODE_ADDRESS_SPENDER_IS_FROZEN;
81     }
82     return _detectTransferRestriction(from, to, value);
83 }
```

Listing 12. Code snippet from README.md

```
488 #### ERC-2980 (Whitelist + Frozenlist)
489
490 Implements the [ERC-2980](https://eips.ethereum.org/EIPS/eip-2980) Swiss
    Compliant Asset Token transfer restriction using two independent address
    lists managed in a single rule:
491
492 * **Whitelist**: only whitelisted addresses may receive tokens. Senders do
```

```
    not need to be whitelisted and may freely transfer tokens they already hold.
493 * **Frozenlist**: frozen addresses are completely blocked — they can neither
    send nor receive tokens.
494 * **Priority**: frozenlist is checked first. If `from` or `to` is frozen,
    the transfer is rejected regardless of whitelist membership.
```

Listing 13. Code snippet from README.md

```
369 | RuleERC2980  | Read-Only | ? | ? | ? | ERC-2980 Swiss Compliant rule
    combining a whitelist (recipient-only) and a frozenlist (blocks both sender
    and recipient). Frozenlist takes priority over whitelist. |
```

Impact in production:

- Operators and integrators relying on docs may expect only `from`/`to` to be frozen-gated. `transferFrom` calls can fail unexpectedly when the spender is frozen, leading to confusion, false bug reports, or operational delays.

Exploit Scenario

Actors and setup:

- Alice: token issuer using RuleERC2980.
- Bob: recipient.
- Charlie: exchange/escrow acting as spender via `transferFrom`.

Scenario:

1. Alice reads the docs and understands that the frozenlist blocks only senders and recipients.
2. Alice freezes Charlie for compliance review but does not anticipate an impact on allowance-based flows.
3. Alice authorizes Charlie to move 100 tokens from her account and attempts `transferFrom(Alice, Bob, 100)` via Charlie.
4. The call fails with code 62 because `_detectTransferRestrictionFrom` rejects

a frozen spender, contrary to Alice's expectation from the docs.

Recommendation

Align documentation with code behavior so integrators correctly anticipate spender-based freezes in `transferFrom`:

- In README's ERC-2980 section, explicitly add the spender to the frozenlist scope and priority description (e.g., "If `from`, `to`, or `spender` is frozen, the transfer is rejected").
- In the summary table entries in README, AGENTS.md, and CLAUDE.md, update the description to "frozenlist blocks sender, recipient, and spender (for transferFrom)".
- Optionally add an inline note in `RuleERC2980Base` NatSpec for `_detectTransferRestrictionFrom` documenting that code 62 is returned when the spender is frozen.
- Add a test vector demonstrating that freezing the spender blocks `transferFrom` while direct `transfer` remains unaffected.

Go back to Findings Summary

I2: hasRole override: DEFAULT_ADMIN_ROLE implicitly passes all role checks, making revoke/renounce ineffective and events misleading

Impact:	Info	Confidence:	High
Target:	AccessControlModuleStandalone.sol	Type:	Logic error

Description

The module overrides function `hasRole` so that any account holding `DEFAULT_ADMIN_ROLE` is treated as having every role. This diverges from OpenZeppelin semantics and makes role management operations misleading, because OpenZeppelin's `grantRole`, `revokeRole`, and `renounceRole` rely on `hasRole` to decide whether to modify `_roles[role].hasRole[account]` and to emit events.

Listing 14. Code snippet from src/modules/AccessControlModuleStandalone.sol

```
41 function hasRole(
42     bytes32 role,
43     address account
44 )
45     public
46     view
47     virtual
48     override(AccessControl, IAccessControl)
49     returns (bool)
50 {
51     // Dev note: default admin is treated as having all roles but may not
   appear in enumerable role members.
52     // The Default Admin has all roles
53     if (AccessControl.hasRole(DEFAULT_ADMIN_ROLE, account)) {
54         return true;
55     } else {
56         return AccessControl.hasRole(role, account);
```

```
57     }
58 }
```

OpenZeppelin's internals gate role state changes and event emission on `hasRole`, so the override above prevents accurate state transitions for admins.

Listing 15. Code snippet from lib/openzeppelin-contracts/contracts/access/AccessControl.sol

```
197 function _grantRole(
198     bytes32 role,
199     address account
200 ) internal virtual returns (bool) {
201     if (!hasRole(role, account)) {
202         _roles[role].hasRole[account] = true;
203         emit RoleGranted(role, account, _msgSender());
204         return true;
205     } else {
206         return false;
207     }
208 }
209
210 function _revokeRole(
211     bytes32 role,
212     address account
213 ) internal virtual returns (bool) {
214     if (hasRole(role, account)) {
215         _roles[role].hasRole[account] = false;
216         emit RoleRevoked(role, account, _msgSender());
217         return true;
218     } else {
219         return false;
220     }
221 }
```

Technical impact in this codebase:

- `revokeRole` and `renounceRole` emit `RoleRevoked` for an admin but do not actually remove effective privileges, since `hasRole` continues to return `true` via the override.

- `grantRole` to an admin becomes a no-op with no `RoleGranted` event, and the admin is not added to enumerable membership.
- `AccessControlEnumerable` queries such as `getRoleMember` and `getRoleMemberCount` omit the admin from non-default roles because `_roles[role].hasRole[admin]` was never set to `true`, while `hasRole` still returns `true`.
- Any function guarded by modifier `onlyRole(<R>)` in contracts inheriting `AccessControlModuleStandalone` is callable by a `DEFAULT_ADMIN_ROLE` holder regardless of explicit membership in `<R>`, even after a `revokeRole` or `renounceRole` attempt.

Affected elements:

- Function `hasRole` in contract `AccessControlModuleStandalone`.
- Functions `grantRole`, `revokeRole`, `renounceRole`, and internals `_grantRole`, `_revokeRole` in OpenZeppelin `AccessControl` relied upon by this module.
- State variable `_roles[role].hasRole[account]` that backs enumeration in `AccessControlEnumerable` and diverges from effective privileges for admins.

Realistic production impact:

- Off-chain monitoring, analytics, and governance automation that trust `RoleGranted` and `RoleRevoked` events or enumerable membership can be misled into believing an admin lost or gained privileges when the effective authorization has not changed.
- Admins cannot granularly renounce individual roles while retaining `DEFAULT_ADMIN_ROLE`, which can break operational procedures or internal controls expecting role separation.
- Security reviews and incident response relying on role event logs will draw incorrect conclusions about who can execute privileged functions guarded by `onlyRole`.

Exploit Scenario

Participants:

- Alice holds `DEFAULT_ADMIN_ROLE` and is initially assigned `OPERATOR_ROLE`.
- Bob operates an off-chain monitor that trusts `RoleGranted`/`RoleRevoked` events and `getRoleMember` for access dashboards.

Steps:

1. Alice calls function `renounceRole(OPERATOR_ROLE, Alice)` on the contract.
2. OpenZeppelin `renounceRole` calls internal `_revokeRole`, which uses `hasRole` to decide whether to clear `_roles[OPERATOR_ROLE].hasRole[Alice]`. Because `hasRole` is overridden to return `true` for any default admin, `_revokeRole` sets the mapping to `false` and emits `RoleRevoked(OPERATOR_ROLE, Alice, Alice)`.
3. Bob observes the `RoleRevoked` event and updates monitoring to reflect that Alice no longer has `OPERATOR_ROLE`. Enumeration queries `getRoleMemberCount(OPERATOR_ROLE)` and `getRoleMember(OPERATOR_ROLE, i)` also no longer list Alice, reinforcing the belief that Alice lost the role.
4. Later, Alice calls a function protected by modifier `onlyRole(OPERATOR_ROLE)`.
5. The modifier checks `hasRole(OPERATOR_ROLE, Alice)`. The override returns `true` because Alice still holds `DEFAULT_ADMIN_ROLE`, so the call succeeds and Alice keeps full operator privileges despite the earlier `RoleRevoked` event and her attempted renunciation.

Variant:

- Governance calls `revokeRole(OPERATOR_ROLE, Alice)`. The same sequence occurs: `RoleRevoked` is emitted, enumeration excludes Alice, but `onlyRole(OPERATOR_ROLE)` still allows Alice to execute because of the `hasRole` override.

Recommendation

Preferred fix:

- Remove the override of function `hasRole` in contract `AccessControlModuleStandalone` so `hasRole` reflects explicit membership only.
- Add a separate helper function `hasPermission(bytes32 role, address account)` that returns `AccessControl.hasRole(role, account) || AccessControl.hasRole(DEFAULT_ADMIN_ROLE, account)`.
- Add a new modifier `onlyRoleOrAdmin(bytes32 role)` that requires `hasPermission(role, msg.sender)`.
- Replace modifier `onlyRole` with `onlyRoleOrAdmin` only where an admin bypass is explicitly intended by business logic.

This preserves correct behavior for `grantRole`, `revokeRole`, `renounceRole`, event emission, and enumerable membership, while still enabling an admin override where desired.

Alternative if the override must remain:

- Override functions `grantRole`, `revokeRole`, and `renounceRole` in `AccessControlModuleStandalone` to base their state changes on `AccessControl.hasRole(role, account)` (the base implementation) rather than the overridden `hasRole`.
- In `revokeRole`, if `AccessControl.hasRole(DEFAULT_ADMIN_ROLE, account)` and `role != DEFAULT_ADMIN_ROLE`, revert with a descriptive error to avoid emitting misleading events.
- In `renounceRole`, if `AccessControl.hasRole(DEFAULT_ADMIN_ROLE, callerConfirmation)` and `role != DEFAULT_ADMIN_ROLE`, revert similarly.
- Update documentation to explicitly state that default admins implicitly pass role checks and cannot be revoked or renounce non-default roles

without first losing `DEFAULT_ADMIN_ROLE`.

Testing:

- Add tests asserting a 1:1 correspondence between `RoleGranted`/`RoleRevoked` events and effective authorization.
- Add tests ensuring `getRoleMember` and `getRoleMemberCount` stay consistent with `hasRole` outcomes for both admin and non-admin accounts.

Go back to Findings Summary

Appendix A: How to cite

Please cite this document as:

Ackee Blockchain Security, Wake Arena AI Report | CMTA: Rules, March 16, 2026 18:00 UTC.

ackee
blockchain security

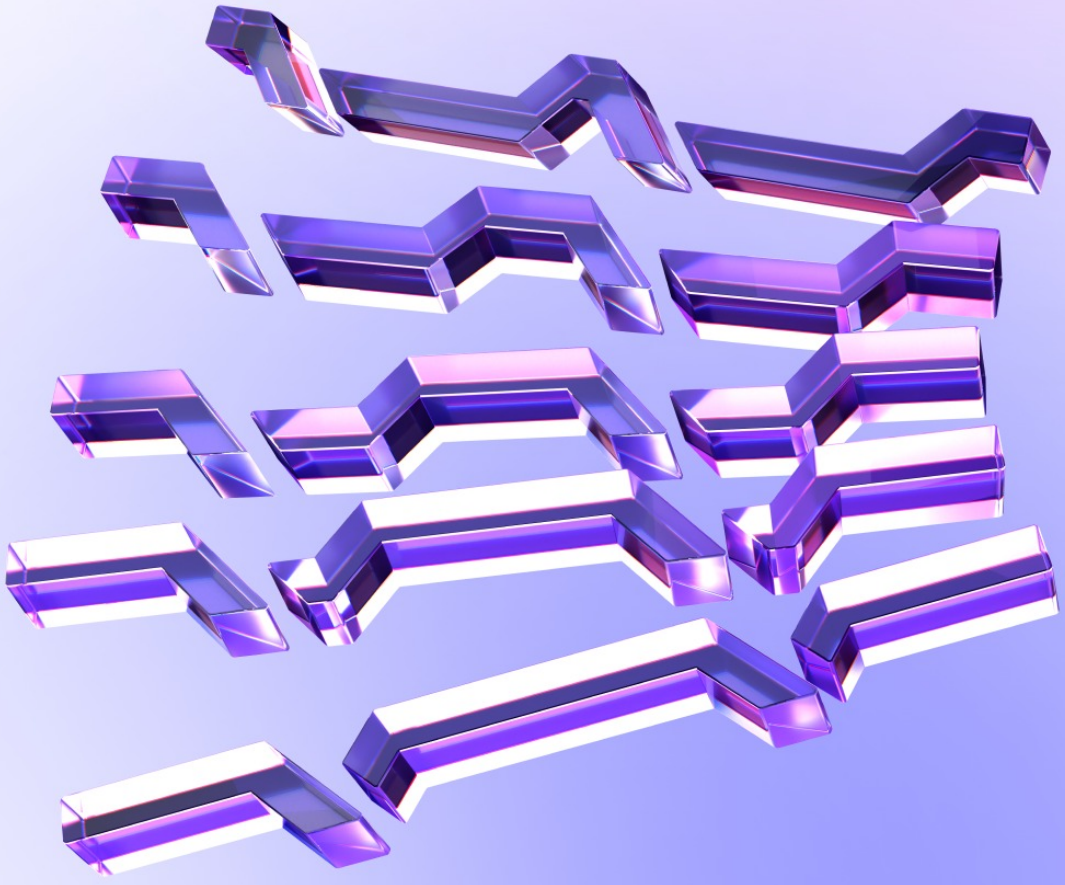
Wake

Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz



Wake Arena AI Report